



National Textile University
Department of Computer Science

Subject

Operating System

Submitted to:

Sir Nasir Mahmood

Submitted by:

Ahmad Farooq

Registration Number

23-NTU-CS-1128

Assignment No.

01

Semester

5th

Section-A: Programming Tasks

Task 1:

```
// Name: Ahmad Farooq
// Rollno: 23-NTU-CS-1128
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <unistd.h>

#define NUM_THREADS 5
void* thread_func(void* arg) {
    int thread_num = *(int*)arg;
    pthread_t tid = pthread_self(); // pthread_self is used to get thread
id
    printf("Thread %d started. Thread ID: %lu\n", thread_num, tid);
    int sleep_time = rand() % 3 + 1; // Built in rand() function is used
to get ransom number
    sleep(sleep_time);
    printf("Thread %d completed after %d seconds.\n", thread_num,
sleep_time);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int thread_nums[NUM_THREADS];
    srand(time(NULL)); // seed for random number is current time to not
get a pattern of numbers
    for (int i = 0; i < NUM_THREADS; i++) {
        thread_nums[i] = i + 1;
        pthread_create(&threads[i], NULL, thread_func, &thread_nums[i]);
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("All threads have finished.\n");
    return 0;
}
```

```
1 // Name: Ahmad Farooq
2 // Rollno: 23-NTU-CS-1128
3 #include <stdio.h>
4 #include <pthread.h>
5 #include <stdlib.h>
6 #include <unistd.h>
7
8 #define NUM_THREADS 5
9 void* thread_func(void* arg) {
10     int thread_num = *(int*)arg;
11     pthread_t tid = pthread_self(); // pthread self is used to get thread id
12     printf("Thread %d started. Thread ID: %lu\n", thread_num, tid);
13     int sleep_time = rand() % 3 + 1; // Built in rand() function is used to get random number
14     sleep(sleep_time);
15 }
16
17 int main() {
18     pthread_t thread;
19     char name[50] = "Ahmad Farooq";
20     printf("Main thread: Waiting for greeting...\n");
21     pthread_create(&thread, NULL, thread_func, name);
22     pthread_join(thread, NULL);
23
24     printf("Main thread: Greeting completed.\n");
25     return 0;
26 }
```

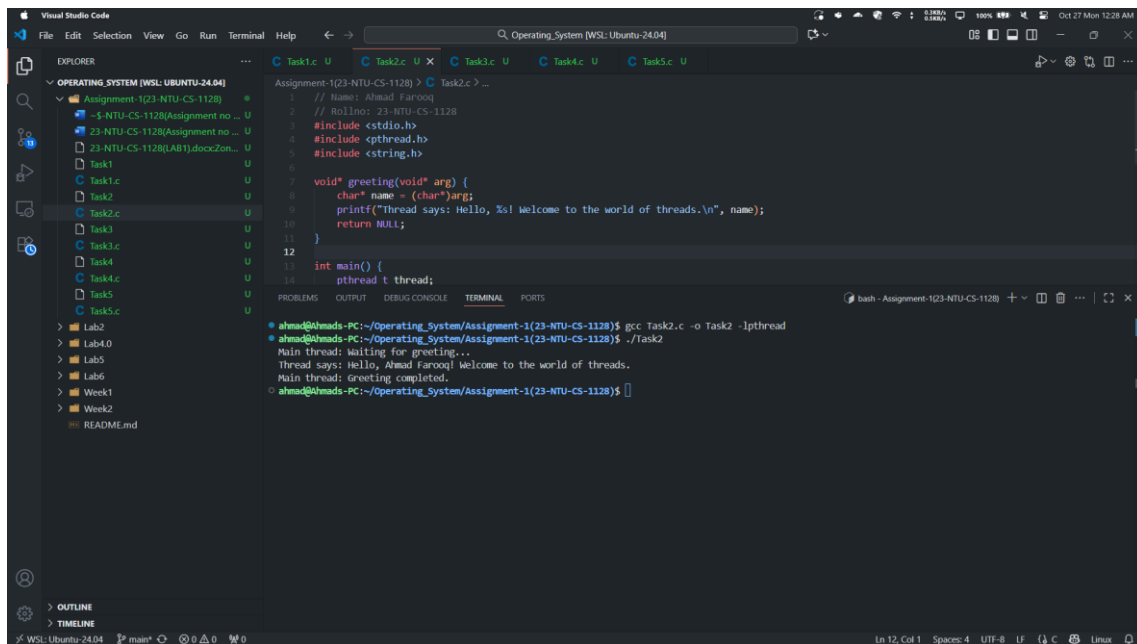
Task 2:

```
// Name: Ahmad Farooq
// Rollno: 23-NTU-CS-1128
#include <stdio.h>
#include <pthread.h>
#include <string.h>

void* greeting(void* arg) {
    char* name = (char*)arg;
    printf("Thread says: Hello, %s! Welcome to the world of threads.\n",
name);
    return NULL;
}

int main() {
    pthread_t thread;
    char name[50]="Ahmad Farooq";
    printf("Main thread: Waiting for greeting...\n");
    pthread_create(&thread, NULL, greeting, name);
    pthread_join(thread, NULL);

    printf("Main thread: Greeting completed.\n");
    return 0;
}
```



Task 3:

```
// Name: Ahmad Farooq
// Rollno: 23-NTU-CS-1128
#include <stdio.h>
#include <pthread.h>

void* number_info(void* arg) {
    int num = *(int*)arg;
    printf("Thread: Number = %d\n", num);
    printf("Thread: Square = %d\n", num * num);
    printf("Thread: Cube = %d\n", num * num * num);
    return NULL;
}

int main() {
    pthread_t thread;
    int num;

    printf("Enter an integer: ");
    scanf("%d", &num);

    pthread_create(&thread, NULL, number_info, &num);
    pthread_join(thread, NULL);

    printf("Main thread: Work completed.\n");
    return 0;
}
```

```
Visual Studio Code
File Edit Selection View Go Run Terminal Help
Operating_System [WSL: Ubuntu-24.04]

EXPLORER
  OPERATING_SYSTEM [WSL: UBUNTU-24.04]
    Assignment-1(23-NTU-CS-1128)
      -5-NTU-CS-1128(Assignment no ... U
      23-NTU-CS-1128(Assignment no ... U
      23-NTU-CS-1128(LAB1)docxZon... U
      Task1
      Task1.c
      Task2
      Task2.c
      Task3
      Task3.c
      Task4
      Task4.c
      Task5
      Task5.c
    Lab2
    Lab4.0
    Lab5
    Lab6
    Week1
    Week2
    README.md

TASK3.c
1 // Name: Ahmad Farooq
2 // Rollno: 23-NTU-CS-1128
3 #include <stdio.h>
4 #include <pthread.h>
5
6 void* number_info(void* arg) {
7     int num = *(int*)arg;
8     printf("Thread: Number = %d\n", num);
9     printf("Thread: Square = %d\n", num * num);
10    printf("Thread: Cube = %d\n", num * num * num);
11    return NULL;
12 }
13
14 int main() {
15     pthread_t thread;
16
17     // ... (code continues) ...
18 }

TERMINAL
ahmad@Ahmad-PC:~/Operating_System/Assignment-1(23-NTU-CS-1128)$ gcc Task3.c -o Task3 -lpthread
ahmad@Ahmad-PC:~/Operating_System/Assignment-1(23-NTU-CS-1128)$ ./Task3
Enter an Integer: 28
Thread: Number = 28
Thread: Square = 784
Thread: Cube = 21952
Main thread: Work completed.
ahmad@Ahmad-PC:~/Operating_System/Assignment-1(23-NTU-CS-1128)$ ./Task3
Enter an Integer: 1128
Thread: Number = 1128
Thread: Square = 1272384
Thread: Cube = 1435249152
Main thread: Work completed.
ahmad@Ahmad-PC:~/Operating_System/Assignment-1(23-NTU-CS-1128)$
```

Task 4:

```
// Name: Ahmad Farooq
// Rollno: 23-NTU-CS-1128
#include <stdio.h>
#include <pthread.h>

long long result = 1; // Global variable to store factorial result

void* factorial(void* arg) {
    int n = *(int*)arg; // Get the number from main thread

    for (int i = 1; i <= n; i++) {
        result *= i; // Calculate factorial
    }

    pthread_exit(NULL); // No need to return anything
}

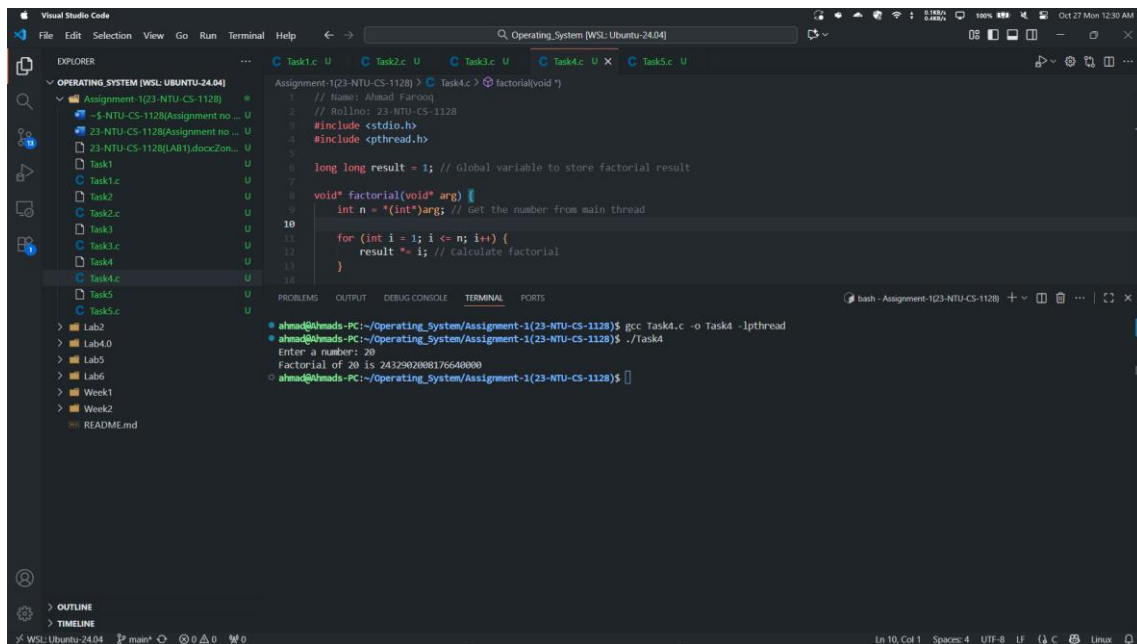
int main() {
    pthread_t thread;
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    pthread_create(&thread, NULL, factorial, &n); // Start thread
    pthread_join(thread, NULL); // Wait for thread to finish

    printf("Factorial of %d is %lld\n", n, result);

    return 0;
}
```



Task 5:

```
// Name: Ahmad Farooq
// Rollno: 23-NTU-CS-1128
#include <stdio.h>
#include <pthread.h>

typedef struct {
    int student_id;
    char name[50];
    float gpa;
} Student;

int d_count = 0; // Shared variable critical part
pthread_mutex_t lock; // mutex lock to prevent race condition

void* student_info(void* arg) {
    Student* s = (Student*)arg;

    printf("\nStudent ID: %d\nName: %s\nGPA: %.2f\n", s->student_id, s->name, s->gpa);

    if (s->gpa >= 3.5) {
        printf("Status: Eligible for Dean's List!\n");
        pthread_mutex_lock(&lock);
        d_count++;
        pthread_mutex_unlock(&lock);
    } else {
        printf("Status: Not eligible for Dean's List.\n");
    }
    return NULL;
}

int main() {
    pthread_t threads[4];
```


Section-B: Short Questions:

2. Define an Operating System in a single line.

Operating system is a layer between hardware and user which allows user to communicate and interact with hardware. Operating system manages resources and helps in executing processes.

3. What is the primary function of the CPU scheduler?

The primary function of CPU scheduler is to select process from queue and assign it to the CPU.

4. List any three states of a process.

Three states of a process is:

1. Ready
2. Running
3. Blocked
4. Exit.

5. What is meant by a Process Control Block (PCB)?

Process control block (PCB) stores information about the processes like its Id, state, registers and much more.

6. Differentiate between a process and a program.

Program	Process
A passive set of instructions stored on disk.(a downloaded software in our C or D disk in Laptop)	An active instance of a program in execution.(a running program)
Static – does not change during storage.	Dynamic – changes as it executes.
Stored in secondary memory like HDD or SSD.	Exists in RAM.

7. What do you understand by context switching?

The process of storing state of an existing running process and loading the state of another process is known as Context Switching. We can also say that the process of multitasking and multiprocessing is all done by context switching.

8. Define CPU utilization and throughput.

The total CPU spent on executing a process is called Cpu utilization. And the number of processes completed in unit time is known as throughput.

9. What is the turnaround time of a process?

The amount of time required to complete a specific process or task is called turnaround Time. Formula for turnaround time is:

$$Tr = \text{Completion time} - \text{Arrival time}$$

10. How is waiting time calculated in process scheduling?

The time spend by a process in the ready queue is known as waiting time and formula to calculate waiting time is:

Waiting Time = Turnaround Time – ServiceTime

11. Define response time in CPU scheduling.

Amount of time it takes between submitting a request and getting first response in known as response time.

12. What is preemptive scheduling?

Preemptive scheduling is the type of process scheduling which allows the cpu to stop a process and start some other process. This is done under some specific conditions. Such algorithms are Round Robin and shortest remaining time first

13. What is non-preemptive scheduling?

Non-preemptive scheduling means once a process starts executing, it cannot be interrupted until it completes or voluntarily releases the CPU. For example algorithms like first come first serve are non-preemptive.

14. State any two advantages of the Round Robin scheduling algorithm.

1. Every process gets equal time of CPU usage.
2. It improves response time in time sharing systems. Because the system will not be stuck on one process if it very time consuming

15. Mention one major drawback of the Shortest Job First (SJF) algorithm.

This can cause problem because system will again and again prefer short tasks to be executed first. What if the most important task was the longest and time taking? Then we will have to wait for all the short tasks to be completed.

16. Define CPU idle time.

CPU idle time is the time when the cpu is not utilized because the process is scheduled but the time is not reach for its execution or there is delay in execution of it then cpu waits till the time come.

17. State two common goals of CPU scheduling algorithms.

One is maximizing the CPU utilization to get maximum output and the Second is to minimize the waiting and turnaround time to execute process as soon as possible.

18. List two possible reasons for process termination.

1. Process completed its execution normally and perfectly.
2. There was an error in the process which didn't allowed to complete its execution and process is exited.

19. Explain the purpose of the wait() and exit() system calls.

1. The purpose of wait() system call is to make parent process to wait until its child process is completely executed which helps in synchronization.
2. And exit() is used to terminate process after it is completed.

20. Differentiate between shared memory and message-passing models of inter-process communication.

Shared Memory	Message Passing
Processes share a common memory area.	Processes exchange messages.
Faster (direct memory access).	Slower (requires kernel intervention).
Must be handled manually.	Handled by the system.

21. Differentiate between a thread and a process.

Thread	Process
A lightweight unit of execution within a process.	An independent program in execution.
Shares memory with other threads in same process.	Has its own separate memory space.
Fast and efficient.	Slow and resource-heavy.

22. Define multithreading.

Multithreading is the ability of a CPU to create multiple threads in a single process to increase efficiency and decrease time utilization. All the threads share same memory from the same process so multithreading is usually faster as compared to multiprocessing.

23. Explain the difference between a CPU-bound process and an I/O-bound process.

CPU-Bound Process	I/O-Bound Process
Requires more CPU calculations and utilization.	Spends more time performing input/output.
Video rendering, encryption.	File reading, printing.
Needs longer CPU bursts.	Needs frequent I/O waits.

24. What are the main responsibilities of the dispatcher?

The dispatcher is responsible for switching the CPU to a new process. It performs context switching and loading selected processes' registers and memory map.

25. Define starvation and aging in process scheduling.

1. Starvation: A process waits indefinitely because the CPU keeps serving higher-priority processes.
2. Aging: A technique that gradually increases the priority of waiting processes to prevent starvation.

26. What is a time quantum (or time slice)?

A time quantum is a fixed unit of time which is used to shift from one process to other preemptively in ROBIN ROUND algorithm. It is assigned to each process and after it the cpu scheduler changes the process.

27. What happens when the time quantum is too large or too small?

- If too large, the system behaves like FCFS (poor response time).
- If too small, context switching overhead increases, reducing efficiency.

28. Define the turnaround ratio (TR/TS).

The turnaround ratio help us to know how efficient and fast an algorithm is because the smaller the value of tr/ts ratio is the better the algorithm is.

29. What is the purpose of a ready queue?

A ready queue stores all processes that are ready and waiting to use the CPU. The scheduler picks processes from this queue based on the chosen scheduling algorithm.

30. Differentiate between a CPU burst and an I/O burst.

CPU Burst	I/O Burst
Time spent executing on CPU.	Time spent waiting for I/O operations.
CPU.	I/O devices.
It is generally shorter.	It can be longer.

31. Which scheduling algorithm is starvation-free, and why?

Round Robin Scheduling is starvation-free because every process gets equal CPU time in cyclic order, preventing indefinite waiting.

32. Outline the main steps involved in process creation in UNIX.

1. Parent process calls fork() to create a child.
2. The child receives a copy of the parent's data and code.
3. The child can call exec() to replace its code with a new program.
4. The parent uses wait() to synchronize with the child's termination.

33. Define zombie and orphan processes.

- **Zombie Process:** A process that has completed execution but still has an entry in the process table because the parent hasn't read its exit status.
- **Orphan Process:** A process whose parent has terminated before it finishes; such processes are adopted by the "init" process.

34. Differentiate between Priority Scheduling and Shortest Job First (SJF).

Priority Scheduling	Shortest Job First (SJF)
Based on priority value.	Based on CPU burst length.
Can be preemptive or non-preemptive.	Usually non-preemptive.
Can cause starvation of low-priority jobs.	Can cause starvation of long jobs.

35. Define context switch time and explain why it is considered overhead.

Context switch time is the time required to save the current process's state and load the next process's state. It is considered overhead because no useful computation happens during this time the CPU is busy switching, not executing.

36. List and briefly describe the three levels of schedulers in an Operating System.

- **Long-Term Scheduler:** Decides which jobs enter the ready queue from the job pool.
- **Short-Term Scheduler:** Chooses which ready process should get the CPU next.
- **Medium-Term Scheduler:** Temporarily removes (swaps out) processes from memory to control degree of multiprogramming.

37. Differentiate between User Mode and Kernel Mode in an Operating System.

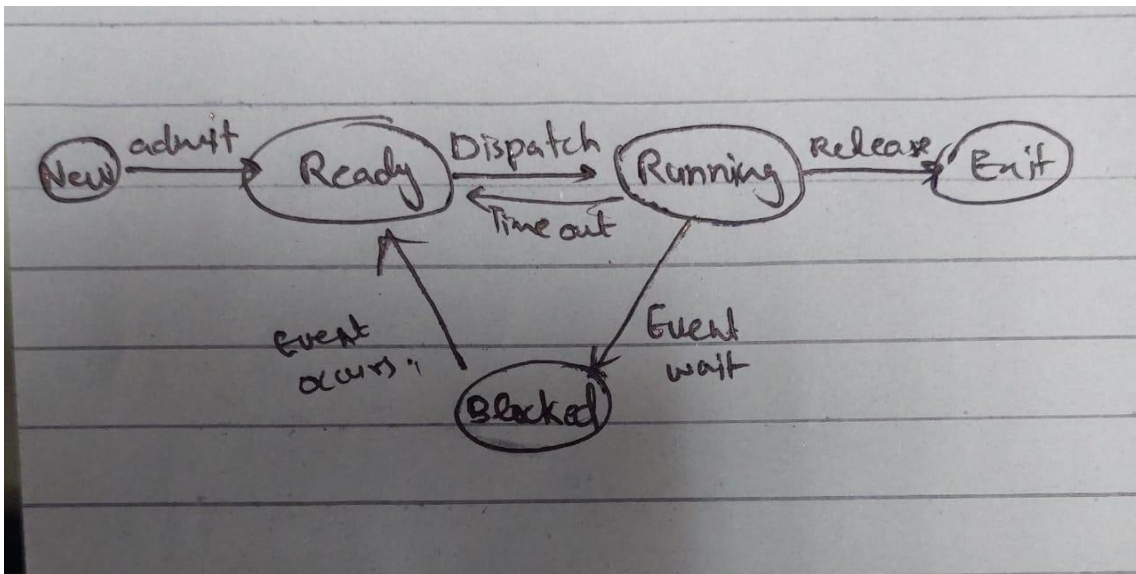
User Mode	Kernel Mode
Limited access to system resources.	Full access to hardware and memory.
Executes user applications.	Executes core OS functions.
Running a text editor.	Running device drivers or system calls.

Section-C: Technical and Analytical Questions

1. Describe the complete life cycle of a process with a neat diagram showing transitions between New, Ready, Running, Waiting, and Terminated states.

- **New:**
This is where a process is just being born. The program has been submitted for execution, but the operating system hasn't yet placed it into the ready queue.
- **Ready:**
Here, the process is all set to run. It's waiting for the CPU to be available. Think of it like being in line — waiting for your turn to use the processor.

- **Running:**
The process is currently using the CPU and performing its operations. Only one process can be in the running state on a single processor at a time.
- **Waiting (or Blocked):**
Sometimes, a process needs to wait for something like user input or a file to finish reading. In that case, it goes to the waiting state until that event happens.
- **Terminated (or Exit):**
Once the process finishes its task or is stopped by the system, it moves to the terminated state. The operating system then releases its resources.



2. Write a short note on context switch overhead and describe what information must be saved and restored.

A context switch happens when the CPU switches from one process to another. During this switch, the operating system has to save the current process's information so that it can continue later from where it left off, and then load the next process's saved information. This saving and loading take a small amount of time, which is called context **switch** overhead. It doesn't do any useful work during that time that's why it's considered an overhead.

Saved Information	Description
Program Counter	Where the process left off (the next instruction to execute)
CPU Registers	Current values inside CPU registers
Stack Pointer	Points to the current stack location of the process
Memory Management Info	Details about which memory the process uses
Process State	The process was ready, waiting, or running

3. List and explain the components of a Process Control Block (PCB).

Here are the Components of the PCB:

Component	Description
Process ID (PID)	A unique number assigned to each process
Process State	Current status (New, Ready, Running, Waiting, Terminated)
CPU Registers	Saved values of registers when process is not running
Program Counter	The address of the next instruction to execute
Memory Information	Base and limit addresses of the process in memory
Accounting Information	Details like CPU usage, time limits, etc.
I/O Status	List of I/O devices allocated to the process

4. Differentiate between Long-Term, Medium-Term, and Short-Term Schedulers with examples.

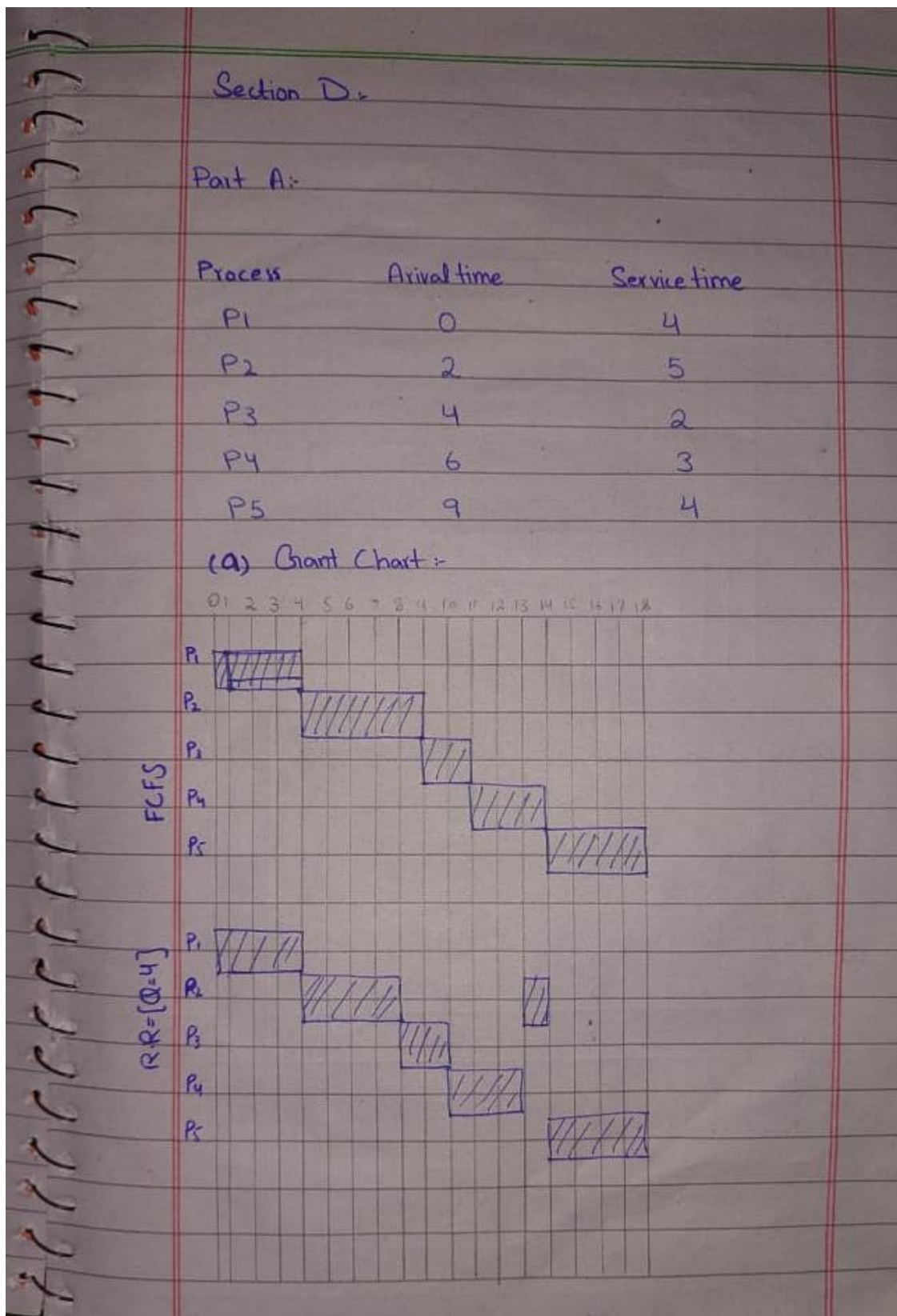
Type of Scheduler	Description	Example
Long-Term Scheduler	Decides which processes should be loaded into memory for execution. It controls the degree of multiprogramming (how many processes are in memory).	When you open several apps, only some are actually loaded into memory.
Medium-Term Scheduler	Temporarily removes processes from memory and puts them in secondary storage to free space. Later, it can bring them back.	When your system suspends a process because of low memory.
Short-Term Scheduler	Decides which process in the ready queue will run next on the CPU.	CPU chooses which ready process to execute next.

5. Explain CPU Scheduling Criteria (Utilization, Throughput, Turnaround, Waiting, and Response) and their optimization goals.

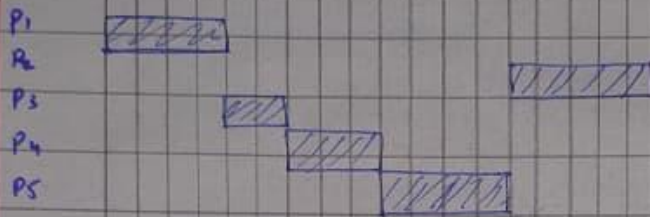
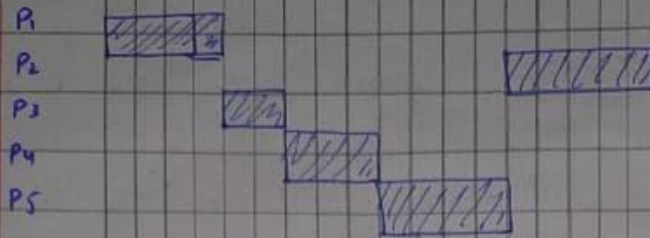
Criterion	Description	Goal
CPU Utilization	This simply means how much the CPU is being used instead of sitting idle. A good operating system tries to keep the CPU busy all the time by switching between processes efficiently. The higher the CPU utilization, the better the performance of the system.	Maximize
Throughput	Throughput measures how many processes are fully completed within a certain time period. It's like how many tasks your computer finishes every minute. If the system finishes more processes in less time, it means the scheduling is working well.	Maximize
Turnaround Time	Turnaround time is the total time a process spends in the system from the moment it is submitted until it finishes completely. It includes waiting time, execution time, and any delays. In short, it's how long a user has to wait to get the result of their task.	Minimize
Waiting Time	This is the amount of time a process spends waiting in the ready queue before it gets the CPU. Too much waiting time can make the system feel slow and unresponsive. A good scheduling algorithm reduces waiting time by balancing process priorities and arrival times.	Minimize
Response Time	Response time is the time between when a process is submitted and when it starts giving its first response. It's especially important for systems where users interact directly, like clicking buttons or typing commands. A shorter response time makes the system feel faster and more responsive.	Minimize

Section-D: CPU Scheduling Calculations

Part A



0 2 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19



	Process	Finish	Finish - Arrival Turn Around	Service Time (Ts)	Tr - Ts Waiting	Tr / Ts
FCFS	P ₁	4	4	4	0	1
	P ₂	9	7	5	2	1.4
	P ₃	11	7	2	5	3.6
	P ₄	14	8	3	5	2.67
	P ₅	18	9	4	5	2.25
	<u>Avg</u>		7	4	3.4	2.18
RR: [Q=4]	P ₁	4	4	4	0	1
	P ₂	18	16	5	11	3.2
	P ₃	10	6	2	4	3
	P ₄	13	7	3	4	2.4
	P ₅	17	8	4	4	2
	<u>Avg</u>		8.2		4.6	2.32

	Process	Finish	T_r	T_s	waiting	T_r/T_s
STF	P_1	4	4	4	0	1
	P_2	18	16	5	11	3.2
	P_3	6	2	2	0	1
	P_4	9	3	3	0	1
	P_5	13	4	4	0	1
	Avg		5.8		2.2	1.44

SRTF	P_1	4	4	4	0	1
	P_2	18	16	5	11	3.2
	P_3	6	2	2	0	1
	P_4	9	3	3	0	1
	P_5	13	4	4	0	1
	Avg		5.8		2.2	1.44

cpu idle for all = 0

Best Algorithm:-

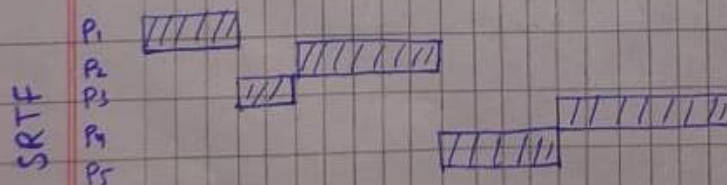
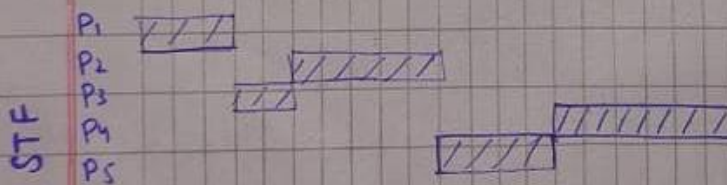
STF/SJF and SRTF
having the shortest time (waiting and
turn around time) are the best and
most suitable.

Part B

Part B:

Process	Arrival time	Service time
P ₁	0	3
P ₂	1	5
P ₃	3	2
P ₄	9	6
P ₅	10	4

Gantt Chart:



FCFS

Process	Finish	Tr	Ts	waitingtime	Tr/Ts
P ₁	3	3	3	0	1
P ₂	8	7	5	2	1.4
P ₃	10	7	2	5	3.5
P ₄	16	7	6	1	1.17
P ₅	20	10	4	6	2.5
Avg		6.8		2.6	1.91

RR [Q=4]

P ₁	3	3	3	0	1
P ₂	10	9	5	4	1.8
P ₃	9	6	2	4	3.0
P ₄	20	11	6	5	1.8
P ₅	18	8	4	4	2
Avg		7.4		3.4	1.92

STF

P ₁	3	3	3	0	1
P ₂	10	9	5	4	1.8
P ₃	5	2	2	0	1
P ₄	20	11	6	5	1.8
P ₅	14	4	4	0	1
Avg		5.8		1.8	1.33

SRTF

P ₁	3	3	3	0	1
P ₂	10	9	5	4	1.8
P ₃	5	2	2	0	1
P ₄	20	11	6	5	1.8
P ₅	14	4	4	0	1
Avg		5.8		1.8	1.33

Cpu idle for all = 0

Best algorithm =

The best algorithm is STF and SRTF because the lowest avg T_r/T_s is provided by these algorithms.

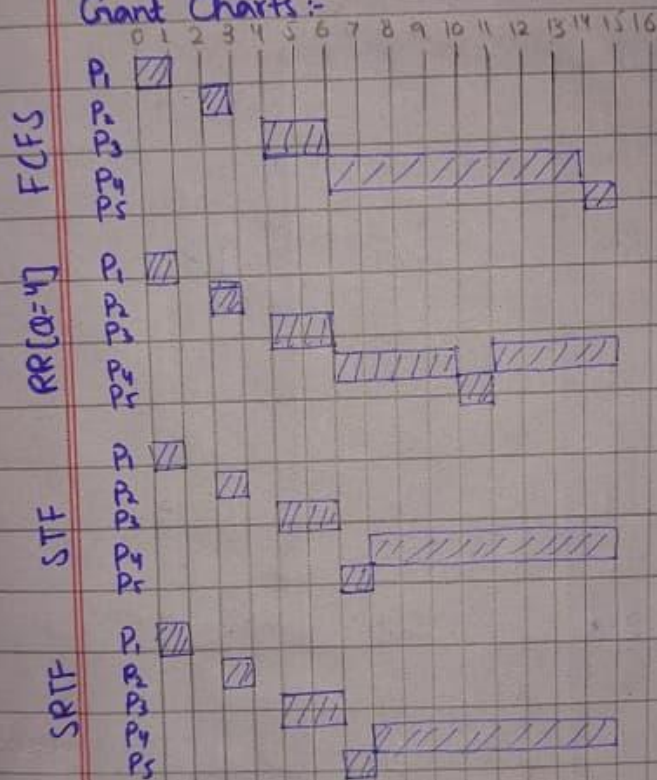
And according to calculation these have least waiting and turnaround avg time.

Part C

Part C- Selecting our own arrival and service time

Process	Arrival time	Service time
P ₁	0	1
P ₂	2	1
P ₃	4	2
P ₄	6	8
P ₅	6	1

Gantt Charts:-



	Process	Finish	Tr	Ts	waiting	Tr/Ts
FCFS	P ₁	1	1	1	0	1
	P ₂	3	1	1	0	1
	P ₃	6	2	2	0	1
	P ₄	14	8	8	0	1
	P ₅	15	9	1	8	9
	Avg		4.2		1.5	2.6
RR [Q=4]	P ₁	1	1	1	0	1
	P ₂	3	1	1	0	1
	P ₃	6	2	2	0	1
	P ₄	15	9	8	1	1.125
	P ₅	11	5	1	4	5
	Avg		3.6		1	1.825
STF	P ₁	1	1	1	0	1
	P ₂	3	1	1	0	1
	P ₃	6	2	2	0	1
	P ₄	15	9	8	1	1.125
	P ₅	7	1	1	0	1
	Avg		2.8		0.2	1.025
SRTF	P ₁	1	1	1	0	1
	P ₂	3	1	1	0	1
	P ₃	6	2	2	0	1
	P ₄	15	9	8	1	1.125
	P ₅	7	1	1	0	1
	Avg		2.8		0.2	1.025

CPU idle time:-

CPU idle time for all algorithms is "2" 1 unit after completion of task 1 and 1 unit after completion of task 2.

Best algorithm:-

The best algorithm for this problem is STF and SRTF because these algorithms have least waiting time and least avg turn around time.

Also minimum avg t_r/t_s is of these two algorithms.